UNIVERSIDAD PRIVADA DEL VALLE

FACULTAD DE INFORMATICA Y ELECTRONICA

CARRERA DE INGENIERIA EN SISTEMAS INFORMÁTICOS



Actualización del Test Plan en un entorno ágil – PedidoHub

Estudiante:

- Luis Andrés Suárez Andrade

Asignatura: Aseguramiento de la Calidad del Software (SQA)

Semestre: Sexto

Grupo: A

Santa Cruz – Bolivia

Proyecto	PedidoHub — gestión de restaurante (ASP.NET Core MVC 8, EF Core, SQL Server)
Versión del plan	2.0 (versión ágil; reemplaza al plan tradicional IEEE 829 v1.0)
Autor / QA Lead	Luis Andrés Suárez Andrade (trabajo individual)
Marco ágil	Kanban con ritmo de sprints de 1 semana (ver 1.2)
Repositorio	`SQA/Segundo Momento/PedidoHub`
Fecha	28 de octubre de 2026

0. Análisis del entorno ágil (Fase 1)

0.1 Marco ágil utilizado

El proyecto se gestiona con **Kanban con cadencia de sprint semanal** y prácticas de **XP** para la calidad: pruebas primero (TDD), integración continua y refactorización. El trabajo fluye por el tablero `Backlog → En análisis → En desarrollo → En pruebas → Hecho`, con **límite de trabajo en curso (WIP) de 3**. Al cierre de cada sprint se hace una revisión de métricas y una retrospectiva.

Se eligió Kanban+XP (y no Scrum puro) porque el equipo real es de **una persona**: no hay ceremonias que dependan de varios roles, pero sí se conserva lo que aporta valor a la calidad: flujo continuo, WIP limitado, Definition of Ready/Done, automatización y mejora continua.

0.2 Roles relevantes para la calidad

Al ser un equipo de una persona, los roles se **ejercen por separado en momentos distintos** (sombreros), y la separación se documenta en cada ítem del tablero (quién "desarrolla" y quién "valida").

Rol	Responsable	Responsabilidad de calidad
Product Owner (PO)	Luis Suárez (sombrero PO)	Prioriza por valor, escribe y valida criterios de aceptación
QA Champion / QA Lead	Luis Suárez (sombrero QA)	Mantiene este plan, métricas y auditoría; decide la salida de un sprint
Developer	Luis Suárez (sombrero Dev)	Implementa, escribe pruebas unitarias, corrige defectos
Tester / Automation Engineer	Luis Suárez (sombrero QA)	Diseña y ejecuta pruebas automatizadas (xUnit, Selenium, Cypress) y exploratorias
Scrum Master / Facilitador	Luis Suárez	Mantiene el tablero, el WIP y la retrospectiva

> ***Nota de independencia:*** *para reducir el sesgo de autorrevisión, la verificación de cada historia se hace con pruebas escritas* ***antes*** *del código (TDD) y se registra la ejecución en rojo y luego en verde (`Docs/evidencias/tdd-fase-roja.txt`).*

0.3 Artefactos del sprint sometidos a pruebas

- Historias de usuario HU-01 a HU-17 (sección 2).
- Código fuente: controladores, servicio de reportes, modelos y vistas.
- Esquema y datos de ejemplo (migración `CrearModeloPedidoHub`).
- Endpoint de operación `/health` y cabeceras HTTP de seguridad.
- Pipeline de CI (`.github/workflows/ci.yml`).

1. Introducción

1.1 Propósito

Este plan define cómo se **gestiona y asegura la calidad de PedidoHub de forma iterativa e incremental**: qué se prueba, con qué herramientas, quién es responsable, cuándo una historia está lista y cómo se mide el resultado. Sustituye al plan tradicional (IEEE 829), que describía las pruebas como una fase al final.

1.2 Integración en el marco ágil

Práctica	Cómo se integra
Calidad desde el inicio	Cada historia nace con criterios de aceptación en formato Gherkin (Dado/Cuando/Entonces)
Shift-left	Las pruebas se escriben antes de la implementación (TDD)
Feedback rápido	Suite unitaria + integración en ~3 s; se ejecuta en cada *push*
Pruebas continuas	CI en GitHub Actions: compila, prueba, mide cobertura y levanta E2E
Responsabilidad compartida	Una historia no pasa a "Hecho" sin pruebas verdes, cualquiera sea quien la implemente
Mejora continua	Retrospectiva por sprint con las métricas de la sección 7

1.3 Revisión del plan original (Fase 2)

Aspecto del plan tradicional	Problema en entorno ágil	Actualización
Pruebas en una fase final	Defectos descubiertos tarde	**Estrategia de pruebas continua** en cada historia
Casos de prueba manuales extensos	Lentos y no repetibles	**Automatización con 2 herramientas: Selenium y Cypress** (+ xUnit para unitarias/integración)
Documento estático	Se desactualiza	Plan versionado en el repo junto al código
Sin enfoque de desarrollo	Pruebas desligadas del diseño	**TDD** (nivel unitario/regla de negocio) y **BDD** (criterios Gherkin, nombres Dado/Cuando/Entonces)
Entregas grandes	Integración tardía	**CI/CD** con ejecución en cada cambio
Todo se prueba por igual	Esfuerzo mal distribuido	**Priorización por valor** (sección 3.4)

2. Alcance

Funcionalidades y historias cubiertas en el sprint (fuente: README del proyecto y reglas de negocio implementadas).

ID	Historia de usuario	Prioridad	Criterio de aceptación (resumen)
HU-01	Como mesero, quiero ver el plano de mesas para saber cuáles están libres	Alta	Se muestran las 5 mesas con su estado
HU-02	Como mesero, quiero abrir un pedido en una mesa libre en un paso	Alta	No se permite abrir otro pedido en una mesa con pedido sin facturar
HU-03	Como mesero, quiero agregar platos a un pedido	Alta	Si el plato ya existe, se suma la cantidad; solo pedidos abiertos/servidos
HU-04	Como cajero, quiero facturar un pedido	Alta	NIT de 5–20 dígitos; no se factura dos veces; el pedido pasa a *Facturado*; la fecha de emisión no es anterior al pedido
HU-05	Como cajero, quiero anular una factura	Media	El pedido vuelve a *Servido*
HU-06	Como recepcionista, quiero reservar mesas	Alta	Personas ≤ capacidad; sin superposición < 2 h; **no se aceptan fechas pasadas**
HU-07	Como administrador, quiero gestionar clientes y teléfonos	Media	Todo cliente tiene ≥ 1 teléfono; no se borra el último
HU-08	Como administrador, quiero gestionar platos	Media	Nombre único; precio 0.01–10000; no se elimina un plato usado
HU-09	Como gerente, quiero el reporte de ventas por período	Alta	Filtro por fechas y empleado; total y ticket promedio correctos
HU-10	Como gerente, quiero ver los platos más vendidos	Media	Solo pedidos facturados; orden por unidades y luego nombre
HU-11	Como operador, quiero un endpoint de salud	Media	`/health` responde `Healthy` si la BD está disponible
HU-12	Como usuario, quiero que la aplicación sea segura por defecto	Alta	Antiforgery en POST; cabeceras `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`
HU-13	Como administrador, quiero iniciar sesión y que cada rol vea solo lo suyo	Alta	Login con clave hasheada; sin sesión → login; mesero sin acceso a Empleados/Facturas/Reportes/Calidad; cookie HttpOnly; sin redirección abierta
HU-14	Como administrador, quiero una bitácora de quién cambió qué	Media	Cada alta, cambio y baja registra usuario, fecha, entidad y valores anterior → nuevo
HU-15	Como administrador, quiero un panel con las métricas de calidad	Media	KPIs, pirámide de pruebas, cobertura por archivo, defectos, trazabilidad y DoD
HU-16	Como visitante, quiero una página de presentación del sistema	Baja	La raíz `/` es pública, muestra funciones, roles e indicadores de calidad
HU-17	Como gerente, quiero exportar los reportes a CSV	Media	Archivo `text/csv` con BOM, filtros respetados, fechas inválidas → 400, neutraliza fórmulas de Excel

Fuera de alcance (este sprint): paginación de los listados, pruebas de carga, internacionalización. Se registran como deuda en la sección 8 y en el backlog.

3. Estrategia de pruebas

3.1 Pirámide de pruebas

```
           /  E2E  \        Selenium (C#) + Cypress (JS)   — flujos críticos, 36 escenarios
          /----------\
         / Integración \    xUnit + WebApplicationFactory + SQLite — reglas de negocio vía
HTTP
        /----------------\
       /     Unitarias    \  xUnit — validaciones, cálculos, ReporteService
```

3.2 Tipos de prueba y herramientas

Tipo	Qué cubre	Herramienta	Automatizada
Unitarias	Validaciones de modelos, totales, métricas de reportes, consultas de `ReporteService`	xUnit	Sí
Integración	Controladores + vistas + EF Core con BD SQLite en memoria; antiforgery; redirecciones	xUnit + `WebApplicationFactory` (134 pruebas) y unitarias (46)	Sí
Aceptación (BDD)	Reglas de negocio redactadas como Dado/Cuando/Entonces	xUnit (nomenclatura Gherkin)	Sí
End-to-end	Flujos de usuario reales en navegador	**Selenium WebDriver** (Chrome) y **Cypress**	Sí
Regresión	Toda la suite en cada cambio	CI (GitHub Actions)	Sí
Exploratorias	Sesiones de 30 min con *charters* (sección 5.3)	Manual	No
Seguridad básica	Antiforgery, cabeceras, no exponer trazas	xUnit + Cypress	Parcial
Humo	Todas las pantallas responden 200	xUnit (`NavegacionTests`)	Sí

3.3 TDD, BDD y ATDD

- **TDD:** para HU-04, HU-06, HU-02 (edición/creación), HU-11 y HU-12 se escribieron primero las pruebas. La ejecución inicial quedó **en rojo (5 fallos esperados de 96)** y tras implementar pasó a **verde (96/96 en ese momento; 216/216 al cierre del sprint con las pruebas añadidas después)**. Evidencia: `Docs/evidencias/tdd-fase-roja.txt`.
- **BDD:** los criterios se redactan en Gherkin y se codifican con nombres `Dado_…_Cuando_…_Entonces_…` (`ReglasDeNegocioTests`, `FlujosDeUsuarioTests`, `pedidohub.cy.js`).

- **Nota sobre HU-13 a HU-17:** se construyeron después y sus pruebas de aceptación (`SeguridadTests`, `BitacoraTests`, `ReportesCsvTests`, E2E-12 a E2E-17 y el bloque "Acceso, landing y roles" de Cypress) se escribieron al cerrar cada incremento, no antes del código. El ciclo rojo→verde documentado corresponde a DEF-01 a DEF-05.
- **ATDD:** el PO define el criterio de aceptación **antes** de la historia; la prueba E2E correspondiente es el criterio ejecutable.

Ejemplo (HU-06):

```
Escenario: Reserva en el pasado
  Dado un cliente "Ana Torres" y la mesa 2
  Cuando registro una reserva con fecha 10/01/2026
  Entonces el sistema la rechaza con "La fecha de una reserva nueva no puede estar en el
pasado."
```

3.4 Priorización de pruebas según valor del producto

Prioridad	Criterio	Historias	Cuándo se ejecuta
P1 — crítica	Impacta ingresos o dinero (pedidos, facturación)	HU-02, 03, 04, 05, 09	Cada *push* (unit+integración) y E2E en cada PR
P2 — alta	Operación diaria	HU-01, 06, 07, 08, 12	Cada *push*
P3 — media	Soporte y conveniencia	HU-10, 11	Cada *push* (barato)

3.5 Automatización con dos herramientas

- **Selenium WebDriver (C#/xUnit):** proyecto `PedidoHub.E2E`, 17 escenarios (17/17 pasan), Chrome *headless*.
- **Cypress (JavaScript):** carpeta `cypress/`, 19 escenarios (19/19 pasan), reporte JUnit.
- Se complementan: Selenium aprovecha el ecosistema .NET y comparte datos de prueba; Cypress ofrece ciclo rápido de depuración y verificación HTTP directa.

3.6 Integración continua (CI/CD)

Archivo `.github/workflows/ci.yml`, en cada *push* y *pull request*:

- Restaurar y compilar la solución.
- Ejecutar pruebas unitarias e integración con cobertura (coverlet).
- Publicar resultados y cobertura como artefactos.
- Levantar SQL Server (servicio), aplicar migraciones, iniciar la app y ejecutar Selenium y Cypress.
- El *pipeline* falla si hay una prueba en rojo (puerta de calidad).

4. Roles y responsabilidades

Rol	Responsabilidades en este plan	Evidencia
QA Champion	Mantener el plan, métricas, auditoría interna, decidir salida del sprint	Este documento, informe de auditoría
Developers	Pruebas unitarias, corregir defectos, mantener el CI en verde	`PedidoHub.Tests`
Product Owner	Escribir y validar criterios de aceptación; aceptar resultados	Tabla de historias (sección 2)
Scrum Master	Facilitar integración del plan en el tablero y la retrospectiva	`Docs/TableroKanban.md`
Tester/Automation	Diseñar y ejecutar automatizadas y exploratorias	`PedidoHub.E2E`, `cypress/`

5. Ciclo de vida de las pruebas dentro del sprint

5.1 Flujo por historia

```
Backlog → (DoR) → Escribir criterios + pruebas (rojo) → Implementar (verde) → Refactor
       → CI (unit+integración+E2E) → Revisión QA → (DoD) → Hecho → Retro / métricas
```

5.2 Pruebas automatizadas vs manuales

Momento	Automatizadas	Manuales
Desarrollo	Unitarias y de integración al guardar/commit	—
Integración	Suite completa en CI	—
Fin de historia	E2E de la historia	Revisión de UX/accesibilidad
Cierre de sprint	Regresión completa	Sesión exploratoria + auditoría

5.3 Pruebas exploratorias (charters)

- *Explorar* el plano de mesas *con* pedidos simultáneos *para descubrir* inconsistencias de estado.
- *Explorar* el formulario de reservas *con* valores límite de fecha y capacidad *para descubrir* validaciones faltantes.
- *Explorar* la facturación *con* pedidos modificados *para descubrir* totales incorrectos.

5.4 Gestión de defectos

Estados: Nuevo → Confirmado → En corrección → Verificado → Cerrado. Severidad (Crítica/Mayor/Menor) y prioridad se registran en el tablero. Todo defecto corregido recibe una **prueba de regresión**.

6. Criterios de entrada y salida

6.1 Definition of Ready (DoR) — para iniciar una historia

- ☐ Redactada como "Como… quiero… para…".
- ☐ Criterios de aceptación en Gherkin.
- ☐ Prioridad asignada por el PO.
- ☐ Datos de prueba identificados.
- ☐ Estimación menor a 1 sprint.

6.2 Definition of Done (DoD) — para cerrar una historia

- ☐ Pruebas escritas **antes** del código (TDD) y en verde.
- ☐ Cobertura de líneas del código nuevo ≥ 80 %.
- ☐ Cero defectos críticos/mayores abiertos de la historia.
- ☐ Criterios de aceptación verificados (automatizados o manuales).
- ☐ CI en verde.
- ☐ Documentación actualizada (README / este plan).

6.3 Criterios de salida del sprint

- 100 % de pruebas P1 y P2 pasando.
- Pass rate global ≥ 98 %.
- Sin defectos críticos abiertos.
- Auditoría interna realizada con acciones correctivas registradas.

7. Métricas y control de calidad

7.1 Resultados del sprint (medidos)

Métrica	Fórmula	Valor	Meta
Pruebas automatizadas totales	xUnit 180 + Selenium 17 + Cypress 19	**216**	—
Pass rate final	pasadas / totales	**216 / 216 = 100 %**	≥ 98 %
Fail rate final	falladas / totales	0 %	≤ 2 %
Pass rate en fase roja TDD	pasadas / totales	91 / 96 = 94.8 % (5 fallos esperados)	—
Defectos encontrados	auditoría + TDD	**6** (1 crítico, 3 mayores, 2 menores)	—
Defectos corregidos	corregidos / encontrados	**6 / 6 = 100 %**	100 %
Cobertura de líneas (lógica de la app)	coverlet	**1328 / 1633 = 81.3 %**	≥ 80 % (DoD)
Cobertura de `ReporteService`	coverlet	100 %	≥ 80 %

Métrica	Fórmula	Valor	Meta
Defect density	defectos / KLOC (1.633 KLOC ejecutables)	**3.7 defectos/KLOC**	< 5
Tiempo de la suite unit.+integración	—	≈ 13 s	< 30 s
Throughput (velocidad)	tarjetas terminadas por sprint	**23** (17 historias + 6 defectos)	—

Estas métricas se publican dentro de la propia aplicación en el **Panel de calidad** (`/Calidad`, solo administrador), que lee `PedidoHub/Data/calidad.json`; ese archivo lo actualiza `scripts/cobertura.ps1` a partir de los resultados reales de las pruebas. Detalle por archivo: `Docs/evidencias/cobertura.md`. Quedan por debajo de la meta `TelefonosController` (51.4 %) y `EmpleadosController` (60.5 %), registrados como TD-04.

7.2 Definiciones

Métrica	Definición
Cobertura	% de líneas/ramas ejecutadas por las pruebas (coverlet)
Defect density	defectos / KLOC. Con 6 defectos y 1.633 KLOC ejecutables resulta 3.7 por KLOC
Pass/Fail rate	pruebas pasadas o fallidas / totales
Velocidad del equipo	historias terminadas por sprint (Kanban: *throughput* semanal)
Lead time	días desde "En desarrollo" hasta "Hecho"

7.3 Velocidad / throughput

Sprint actual: **23 tarjetas terminadas (17 historias y 6 defectos corregidos)**. Se registrará el *throughput* semanal en el tablero para comparar sprints.

7.4 Mejoras propuestas (indicadores)

- Subir cobertura de ramas de controladores con más pruebas de integración (Telefonos, Mesas, Empleados).
- Agregar análisis estático (analizadores Roslyn) en el CI.
- Agregar paginación y pruebas de rendimiento ligeras.
- Cambiar las claves iniciales de demostración y agregar bloqueo por intentos fallidos de inicio de sesión.

8. Riesgos y mitigación

#	Riesgo	Prob.	Impacto	Mitigación
R1	Equipo de una persona: sesgo de autorrevisión	Alta	Alto	TDD (prueba antes del código), CI como revisor objetivo, auditoría con checklist

#	Riesgo	Prob.	Impacto	Mitigación
R2	Diferencias entre SQLite (pruebas) y SQL Server (producción)	Media	Alto	E2E en CI contra SQL Server real; conversor decimal solo en pruebas; documentado
R3	Pruebas E2E frágiles (tiempos, selectores)	Media	Medio	Esperas explícitas, datos únicos, selectores por `name`
R4	Datos de ejemplo con fechas fijas (2026) vuelven obsoletas las pruebas de reportes	Media	Medio	Fechas fijas de reporte; usar fechas relativas en reservas
R5	Cuentas de demostración con clave inicial conocida	Media	Alto	Login con roles ya implementado (HU-13, claves hasheadas); en un despliegue real se cambia `Seed:Clave` y se agrega bloqueo por intentos
R6	Sin paginación: degradación con muchos registros	Media	Medio	Backlog; prueba de rendimiento
R7	Dependencia de fuentes externas (Google Fonts)	Baja	Bajo	Aloja fuentes localmente (backlog)
R8	Cambio de zona horaria/cultura	Baja	Medio	Cultura fija `en-US` ya configurada

9. Evidencias entregadas

Evidencia requerida	Ubicación
Test Plan actualizado (versión ágil)	`Docs/TestPlan_Agil.md`
Repositorio con pruebas (TDD/BDD/unitarias)	`PedidoHub.Tests/`, `PedidoHub.E2E/`, `cypress/`
Informe de auditoría interna	`Docs/InformeAuditoria.md`
Integración con tablero ágil	`Docs/TableroKanban.md` y `Docs/backlog-trello.csv` (importable a Trello/Jira)
Resultados de pruebas	`Docs/evidencias/` (TRX de xUnit y Selenium, JUnit de Cypress, cobertura, fase roja TDD)
Indicadores y mejoras	Secciones 7 y 8

10. Bibliografía

- Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del Software: Un Enfoque Práctico*. McGraw-Hill.
- Sommerville, I. (2020). *Ingeniería de Software* (10.ª ed.). Pearson.
- ISTQB Foundation Level Syllabus (v4.0, 2023).
- Crispin, L. & Gregory, J. (2014). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley.
- Fowler, M. (2019). *Continuous Integration and Delivery*. ThoughtWorks.